

Applied LLMs for Forecasting and Policy Research

Lecture 4: Neural Networks and Model Estimation

Stephen Hansen
University College London



Introduction

Neural network models have driven many of the important breakthroughs in machine learning and AI over the last ten years.

They were already well-known and popular in the 1990s.

Several interrelated drivers of their recent growth:

1. Availability of large datasets for training
2. Hardware advances: GPU/TPU, cloud infrastructure
3. Software advances: high-level frameworks for model estimation that hide many details from the user
4. Algorithmic advances: better training algorithms

In this lecture, we describe the structure of the simplest class of neural networks: feed-forward neural networks.

Review of Supervised Learning

Begin with observations $(y_i, \mathbf{x}_i)_{i=1}^N$.

Function $f_\theta(\mathbf{x})$ (potentially non-linear) that maps inputs into outputs.

θ is a vector of parameters that determines the shape of the function.

Typical workflow is

1. Split data into training T and test E portions
2. Fit $f_\theta(\mathbf{x})$ on training data to minimize loss function

$$\theta^* = \arg \min_{\theta} \sum_{i \in T} L(f_\theta(\mathbf{x}_i), y_i)$$

3. Evaluate test-set loss

$$\sum_{i \in E} L(f_{\theta^*}(\mathbf{x}_i), y_i)$$

Outline

Basic Architecture of a Neural Network

Estimation

Neural Networks for Model Estimation

Conclusion

A neuron

As usual, suppose that each data observation i has J associated covariates which we stack in $\mathbf{x}_i$.

As in linear regression, we can linearly combine covariates and add a constant:

$$z_i = \theta_0 + \sum_{j=1}^J \theta_j x_{i,j}.$$

In neural network jargon, the $\{\theta_j\}_{j=1}^J$ terms are sometimes called *weights* and θ_0 is called a *bias*.

We can transform the linear combination via an *activation function* ϕ .

Activation Functions

Traditionally:

1. A sigmoid function (logistic regression):

$$\phi(z_i) = \frac{1}{1 + e^{-z_i}}$$

2. A particular limiting case: step function.
3. Hyperbolic tangent.
4. Identity function (linear regression): $\phi(z_i) = z_i$.

Other activation functions have gained popularity recently:

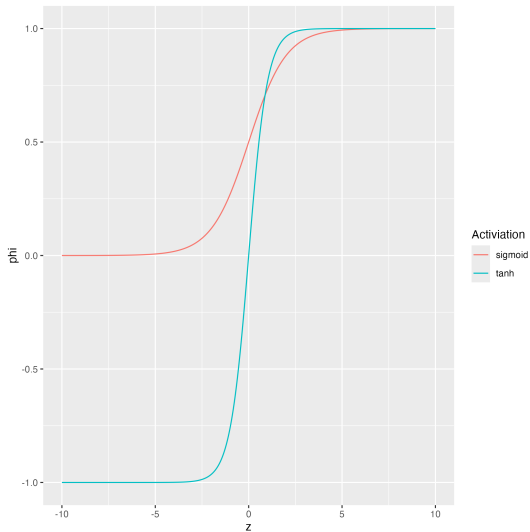
1. Rectified linear unit (ReLU):

$$\phi(z_i) = \max(0, z_i)$$

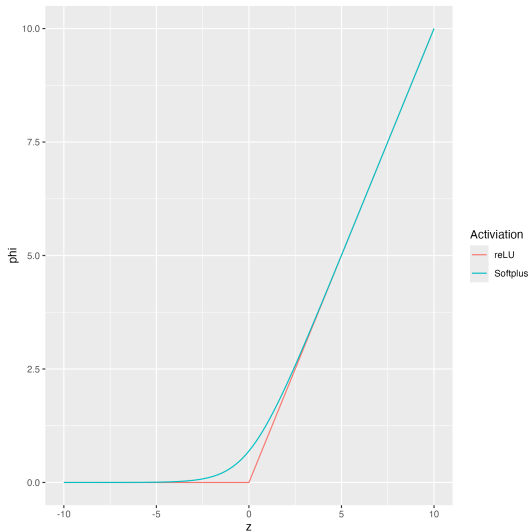
2. Softplus:

$$\phi(z_i) = \log(1 + e^{z_i})$$

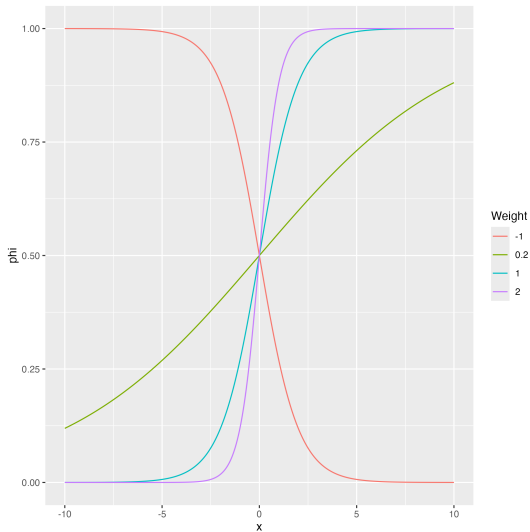
Activation Function Plots I



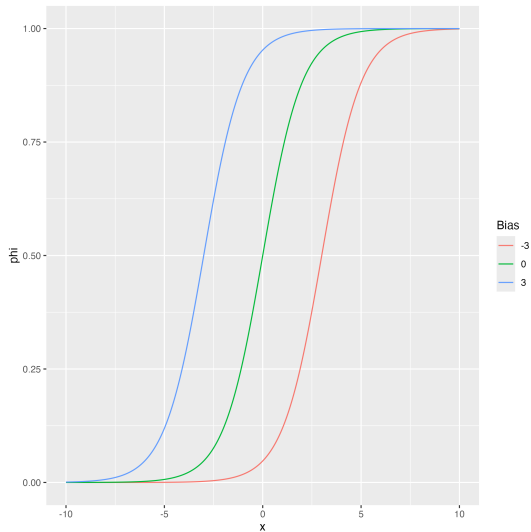
Activation Function Plots II



Effect of Varying Weight (Sigmoid)



Effect of Varying Bias (Sigmoid)



Combining Neurons into a Neural Network

Now suppose we have K separate activation functions operating over the same input $\mathbf{x}_i$.

We first obtain K separate linear combinations of covariates

$$z_{i,k} = \theta_{0,k} + \sum_{j=1}^J \theta_{j,k} x_{i,j} \quad \forall k = 1, \dots, K \quad (\text{Hidden Layer})$$

Next we linearly combine each separate neuron to create a final output

$$f_{\theta}(\mathbf{x}_i) = \theta_0 + \sum_k \theta_k \phi(z_{i,k}) \quad (\text{Output Layer})$$

This is known as a single layer network.

Consists of K non-linear transformations of linear combinations of input variables.

Matrix Notation

We can write the single-layer network compactly in matrix form.

Stack the weights into a $K \times J$ matrix $\mathbf{W}$ and the biases into $\mathbf{b} \in \mathbb{R}^K$:

$$\mathbf{h}_i = \phi(\mathbf{W}\mathbf{x}_i + \mathbf{b}) \quad (\text{Hidden Layer})$$

$$f(\mathbf{x}_i \mid \boldsymbol{\theta}) = \mathbf{w}^\top \mathbf{h}_i + b_0 \quad (\text{Output Layer})$$

where ϕ is applied element-wise and $\mathbf{w} \in \mathbb{R}^K$, $b_0 \in \mathbb{R}$ are the output layer parameters.

Classic Result I

Result

Universal approximation theorem: [Hornik et al., 1989] A neural network with at least one hidden layer can approximate any Borel measurable function mapping finite-dimensional spaces to any desired degree of accuracy.

Why Go Deep?

The universal approximation theorem guarantees that a single hidden layer *suffices* — but with important caveats:

1. The required width K may be **exponentially large** in the input dimension J .
2. Existence $\neq$ efficient learnability. The theorem says nothing about whether we can find the right weights in practice.

An alternative to making a network wider is to make it *deeper*: stack multiple hidden layers so that each layer builds on features learned by the previous one.

In practice, adding layers provides a much more efficient parameterization than adding neurons to a single layer.

Adding Layers

Additional layers can easily be added. Denote the output of layer l as $\mathbf{h}_i^{(l)}$, with $\mathbf{h}_i^{(0)} = \mathbf{x}_i$:

$$\mathbf{h}_i^{(1)} = \phi\left(\mathbf{W}^{(1)}\mathbf{x}_i + \mathbf{b}^{(1)}\right) \quad (\text{Hidden Layer 1})$$

$$\mathbf{h}_i^{(2)} = \phi\left(\mathbf{W}^{(2)}\mathbf{h}_i^{(1)} + \mathbf{b}^{(2)}\right) \quad (\text{Hidden Layer 2})$$

$$\vdots$$

$$f_{\theta}(\mathbf{x}_i) = \mathbf{w}^{(L+1)\top} \mathbf{h}_i^{(L)} + b^{(L+1)} \quad (\text{Output Layer})$$

where $\mathbf{W}^{(l)}$ is a $K_l \times K_{l-1}$ weight matrix, $\mathbf{b}^{(l)} \in \mathbb{R}^{K_l}$ is a bias vector, and ϕ is applied element-wise. For the first hidden layer, $K_0 = J$ since its input is $\mathbf{x}_i \in \mathbb{R}^J$.

Overall structure is called a *feed-forward neural network*.

Multiple layers allow for complex downstream structures to develop by combining simpler upstream structures.

Multi-Valued Output Layer

More generally, the output layer can produce M outputs:

$$\mathbf{f}_{\theta}(\mathbf{x}_i) = \mathbf{W}^{(L+1)}\mathbf{h}_i^{(L)} + \mathbf{b}^{(L+1)} \in \mathbb{R}^M$$

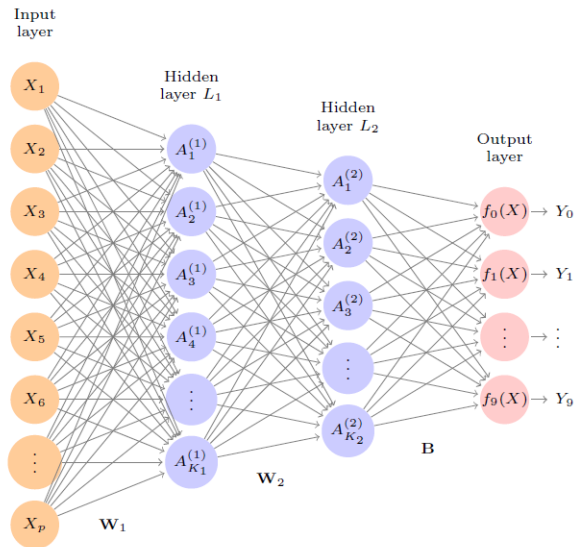
Two common uses:

1. Real-valued outputs: each f_m predicts a different target variable.
2. Class probabilities via the softmax function:

$$P_m(\mathbf{x}_i) = \frac{\exp(f_m)}{\sum_{m'=1}^M \exp(f_{m'})}$$

where M is the number of classes to predict.

Graphical Representation



Why Does Depth Help?

[Montúfar et al., 2014]: a ReLU network with L layers of K neurons each can produce $O(K^L)$ linear regions of the input space, versus $O(K \cdot L)$ for a single hidden layer with the same total neurons.

[Telgarsky, 2016]: there exist functions exactly representable by a depth- L network that cannot even be approximated at depth $L - 1$ without exponentially more neurons.

Bottom line: depth is an exponentially more efficient use of parameters than width.

Outline

Basic Architecture of a Neural Network

Estimation

Neural Networks for Model Estimation

Conclusion

Loss Functions and Gradient Descent

Recall that we want to find $\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$ where $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N L(f_{\theta}(\mathbf{x}_i), y_i)$.

Typical choices for L :

1. Regression (mean squared error): $L = (y_i - f_{\theta}(\mathbf{x}_i))^2$
2. Classification (cross-entropy): $L = - \sum_{m=1}^M y_{i,m} \log P_m(\mathbf{x}_i \mid \theta)$

Gradient descent: repeatedly step in the direction of steepest descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

where $\eta > 0$ is the *learning rate* (a small step size).

Stochastic Gradient Descent

Computing $\nabla_{\theta} \mathcal{L}(\theta)$ requires summing over all N data points, which is expensive for large datasets.

Mini-batch SGD: instead approximate the gradient using a random batch $\mathcal{B}$ of B observations:

$$\theta \leftarrow \theta - \frac{\eta}{B} \sum_{i \in \mathcal{B}} \nabla_{\theta} L(f_{\theta}(\mathbf{x}_i), y_i)$$

This is an unbiased estimate of the full gradient, but much cheaper to compute.

An *epoch* is a single pass through all of the training data.

Automatic Differentiation

SGD requires computing $\nabla_{\theta}\mathcal{L}$, the gradient of the loss with respect to all network parameters.

Since a neural network is a composition of differentiable operations, the chain rule gives an exact gradient. *Automatic differentiation* is the algorithm that evaluates this efficiently.

Modern frameworks (JAX, PyTorch) make this a single function call.

Outline

Basic Architecture of a Neural Network

Estimation

Neural Networks for Model Estimation

Conclusion

From Prediction to Model Estimation

So far we have described neural networks as tools for supervised learning: given labeled data $(y_i, \mathbf{x}_i)$, learn f_θ to predict y from $\mathbf{x}$.

In structural estimation, the problem is different. We want to solve for unknown functions (value functions, policy functions, pricing functions) that satisfy a system of equilibrium conditions.

There is no labeled dataset. Instead, we know the *equations* the solution must satisfy.

Key insight: we can use violations of these equations as the loss function and train a neural network to minimize them.

Recent overviews: [Azinovic et al., 2022]; [Fernández-Villaverde, 2026].

The Problem

Many economic models require solving for an unknown function $p(\mathbf{x})$ that satisfies equilibrium conditions of the form:

$$\mathcal{E}(\mathbf{x}, p(\mathbf{x})) = 0 \quad \text{for all } \mathbf{x} \in \mathcal{X}$$

where $\mathbf{x} \in \mathbb{R}^d$ is a vector of state variables (capital, productivity, wealth, etc.) and $\mathcal{E}$ encodes the model's optimality or equilibrium conditions.

Examples of $\mathcal{E}$:

1. Optimality conditions from dynamic programming: Euler equations (discrete time) or Hamilton-Jacobi-Bellman equations (continuous time)
2. Market clearing conditions in general equilibrium
3. Nash equilibrium

Traditional Approach: Grids

The standard approach is to discretize the state space $\mathcal{X}$ into a grid and solve the equilibrium conditions at each grid point.

This works well in low dimensions, but the number of grid points grows exponentially with the number of state variables d .

Dimensions d	Grid points ($n = 100$)	Feasibility
1	10^2	seconds
2	10^4	minutes
5	10^{10}	infeasible
20	10^{40}	unimaginable

This is the *curse of dimensionality*. It rules out grid-based methods for models with rich heterogeneity or many state variables.

The Neural Network Alternative

Instead of solving on a grid, approximate the unknown function with a neural network:

$$p(\mathbf{x}) \approx f_{\theta}(\mathbf{x})$$

The network takes state variables as inputs and outputs the policy or value function. Its parameters θ are found by training.

Two key properties:

1. The network defines a smooth function *everywhere* on the state space, not just at grid points.
2. The number of parameters grows with the complexity of the network, not with the dimension of the state space.

Equilibrium Errors as the Loss Function

Define the *residual* at a point $\mathbf{x}$ as the equilibrium error when f_{θ} is plugged in for p :

$$\mathcal{R}(\mathbf{x}; \theta) = \mathcal{E}(\mathbf{x}, f_{\theta}(\mathbf{x}))$$

If the network were the exact solution, $\mathcal{R} = 0$ everywhere.

The loss function penalizes violations over randomly sampled points:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{j=1}^B [\mathcal{R}(\mathbf{x}_j; \theta)]^2$$

This is just the mean squared equilibrium error over a mini-batch, which we saw earlier can be minimized with SGD.

Automatic Differentiation Does Double Duty

Recall that automatic differentiation computes *exactly* $\nabla_{\theta}\mathcal{L}$ for SGD.

But many equilibrium conditions $\mathcal{E}$ also involve derivatives of the unknown function *with respect to the state variables*; for example, the HJB equation involves $\partial p/\partial \mathbf{x}$.

When p is a neural network, automatic differentiation gives us these derivatives exactly (no finite differences, no approximation error).

So the same tool that makes training possible also makes it easy to evaluate equilibrium conditions that involve derivatives.

Training Procedure

Putting it all together, the algorithm is:

1. Initialize the neural network f_{θ} .
2. Repeat for many epochs:
 - (a) Sample a batch of state vectors $\{\mathbf{x}_1, \dots, \mathbf{x}_B\}$ from $\mathcal{X}$.
 - (b) Compute the equilibrium residual $\mathcal{R}(\mathbf{x}_j; \boldsymbol{\theta})$ at each point.
 - (c) Update $\boldsymbol{\theta}$ via SGD to reduce $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{B} \sum_j \mathcal{R}(\mathbf{x}_j; \boldsymbol{\theta})^2$.
3. At convergence, f_{θ} approximately satisfies the equilibrium conditions across the entire state space.

This turns *solving* an economic model into *training* a neural network.

Outline

Basic Architecture of a Neural Network

Estimation

Neural Networks for Model Estimation

Conclusion

Conclusion

Neural networks are compositions of simple operations (linear transformation followed by non-linear activation), but depth makes them exponentially more expressive than width alone.

The same machinery that trains them for prediction (SGD + autodiff) also lets us solve economic models: treat equilibrium violations as a loss function and train a network to minimize them.

This is what opens the door to solving heterogeneous agent models with many state variables, where grid-based methods are hopeless.

The accompanying notebook applies the idea to the neoclassical growth model, following [Fernández-Villaverde, 2026].

References I

Azinovic, M., Gaegauf, L., and Scheidegger, S. (2022).

Deep Equilibrium Nets.

[International Economic Review](#), 63(4):1471–1525.

Fernández-Villaverde, J. (2026).

Deep Learning for Solving Economic Models.

[Journal of Economic Literature](#).

Hornik, K., Stinchcombe, M., and White, H. (1989).

Multilayer feedforward networks are universal approximators.

[Neural Networks](#), 2(5):359–366.

Montúfar, G., Pascanu, R., Cho, K., and Bengio, Y. (2014).

On the number of linear regions of deep neural networks.

In [Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2](#), volume 2 of [NIPS'14](#), pages 2924–2932, Cambridge, MA, USA. MIT Press.

Telgarsky, M. (2016).

Benefits of depth in neural networks.

In [Conference on Learning Theory](#), pages 1517–1539. PMLR.